



Ministerio del Poder
Popular para la Educación Universitaria
Universidad Nacional Experimental de las Telecomunicaciones E Informática
Programa Nacional de Formación en Informática
Trayecto III

APLICACIÓN CONSTRUIDA CON FRAMEWORK IONIC

Profeso(a)

Carlos Márquez

Participantes:

Iliana Coronado C.I. V-11.409.705

Mayo 2026

ÍNDICE GENERAL

	Nro.Pág.
INTRODUCCIÓN	1
1. Herramientas necesarias	2
2. Instalación del Entorno de Desarrollo	2
2.1. Node.js	2
2.2. Instalación de Ionic CLI	3
3. Creación del Proyecto	3
3.1. Generar el proyecto desde la consola	3
3.2. Estructura inicial de carpetas generada	4
4. Configuración de los Menús	5
4.1. Reconfiguración completa de rutas	7
5. Desarrollo del contenido de cada página	8
5.1. Página de inicio (home.page.html)	8
5.2. home.page.ts	8
5.3. home.module.ts	10
5.4. home.page.scss	11
5.5. Página de información personal (personal-info.html)	11
5.6. personal-info.page.ts	12
5.7. personal-info.module.ts	13
5.8. personal-info.page.scss	14
5.9. Página de contacto (contact.page.html)	15
5.10. contact.page.ts	16
5.11. contact.module.ts	17
5.12. contact.page.scss	17
5.13. Product-detail	18
5.14. cart	18

6. Paleta y estilos globales	18
6.1. Modificación del archivo variables.scss	18
7. Ejecución y pruebas de la aplicación	20
7.1. Inicio desde el servidor de desarrollo	20
7.2. Prueba de adaptabilidad a dispositivos móviles (responsive)	21
7.3. Verificación de funcionamiento	23
7.4. Visualización de las vistas de la aplicación	24
8. Disponibilidad del Proyecto en GitHub	28
CONCLUSIONES	29
REFERENCIAS	30

INDICE DE FIGURAS

	Nro.Pág
Figura 1 Verificar versión Node.js	2
Figura 2 Verificar versión Ionic	3
Figura 3 Creación del proyecto sweet-dreams-app	3
Figura 4 Estructura inicial del proyecto sweet-dreams-app	4
Figura 5 Código home.page.html	5
Figura 6 Fragmento código app.routing.module.ts	7
Figura 7 Código home.page.ts	9
Figura 8 Código home.module.ts	10
Figura 9 Fragmento código personal-info.html	12
Figura 10 Código personal-info.ts	13
Figura 11 Código personal-info.module.ts	14
Figura 12 Fragmento código contact.page.html	15
Figura 13 Fragmento código contact.page.ts	16
Figura 14 Código contact.module.ts	17
Figura 15 Fragmento código variables.scss	19
Figura 16 Comando ionic serve	20
Figura 17 Compilación exitosa servidor puerto 8100	20
Figura 18 Paso a paso para simular móviles en DevTools	22
Figura 19 Página Home (Inicio)	25
Figura 20 Página Home muestra menú lateral	25
Figura 21 Página Sobre la Chef (personal-info.html)	26
Figura 22 Página Pedidos Especiales (contact.html)	26
Figura 23 Página Detalle (product-detail.html)	27
Figura 24 Página Mi Carrito (cart.html)	27

INDICE DE TABLAS

	Nro.Pág
Tabla 1 Herramientas	2
Tabla 2 Rutas alojadas en app-routing.module.ts	7
Tabla 3 Paleta de colores	19
Tabla 4 Verificación de funcionamiento	23

INTRODUCCIÓN

La Sesión Didáctica 1, viene orientada al estudio y aprendizaje de Ionic un framework de código abierto, que permite el desarrollo de aplicaciones híbridas de fácil integración con Angular, React y Vue, desde su aparición en 2013 ha sido una de las tecnologías líderes en el desarrollo de aplicaciones móviles híbridas. El aprendizaje de las tecnologías móviles representa un hito fundamental en la formación técnica del desarrollador moderno. El uso de Ionic Framework junto a Angular no es solo una elección tecnológica, sino un proceso pedagógico para comprender la convergencia entre el desarrollo web y las capacidades nativas de los dispositivos móviles. El presente trabajo se pone de manifiesto como la parte teórica en relación a este framework hace transición hacia el uso práctico mediante la construcción de una aplicación funcional, donde la modularidad y la inyección de dependencias actúan como pilares para crear software escalable y de alta calidad.

El objetivo de la actividad se centra en crear una aplicación con tres menús o paginas principales: Home, Información Personal y Contacto, Este informe documenta la actividad con el enfoque de un desarrollador que recién empieza a conocer el framework. Desde la instalación del entorno hasta la ejecución final de la aplicación en cada sección se explica el por qué detrás de cada paso.

1. Herramientas necesarias

Para la construcción de la aplicación que hemos denominado **sweet-dreams-app**, nos hemos apoyado en un conjunto de herramientas que permiten llevar adelante el desarrollo de la app.

Tabla 1. Herramientas

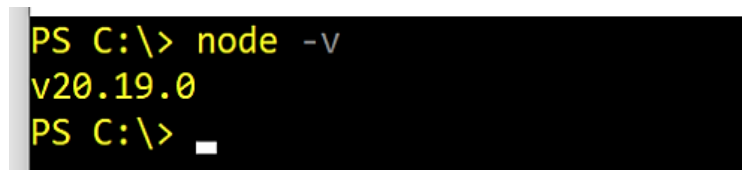
Herramienta	Versión	Propósito en el proyecto
Node.js	18 LTS	Entorno de ejecución de JavaScript; requerido por Ionic y npm
Ionic Framework	8.0.0	Proporciona componentes de interfaz móvil (ion-header, ion-menu, ion-button, etc.) y la integración con Angular.
Angular	20.0.0	Framework estructural: módulos, componentes, servicios, enrutamiento, inyección de dependencias y carga diferida.
Navegador web	Chrome / Edge	Para probar la app con las DevTools en modo móvil
Editor de código	VS Code	Para escribir y organizar el código fuente

Fuente: Elaboración Propia (2026)

2. Instalación del Entorno de Desarrollo

2.1. Node.Js

El entorno de trabajo ya disponía de Node.js 20.0 instalado, por lo que únicamente fue necesario verificar su correcta versión antes de comenzar con el desarrollo.



```
PS C:\> node -v
v20.19.0
PS C:\> _
```

Figura 1. Verificar Versión Node. Js

Fuente: Elaboración Propia (2026)

2.2. Instalación de Ionic CLI

Una vez verificada la versión de Node, se procede a instalar la CLI de ionic de forma global, esto con el fin de que el comando ionic esté disponible desde cualquier ubicación del pc, no limitándolo a un proyecto específico. para ello se uso el comando `npm install -g @ionic/cli`, una vez ejecutado se verifico la correcta instalación consultando la versión de ionic.

```
PS C:\> ionic -v
7.2.1
PS C:\>
```

Figura 2. Verificar Versión Ionic
Fuente: Elaboración Propia (2026)

3. Creación del Proyecto

3.1. Generar el proyecto desde la consola

Con Ionic CLI instalado, el siguiente paso es crear el proyecto. Ionic provee una plantilla llamada 'tabs' que genera automáticamente una aplicación con navegación por pestañas en la parte inferior, que es exactamente lo que se necesita para los tres menús. Finalizada la creación del proyecto, se accede al directorio raíz, donde encontraremos la estructura de carpetas. Es la base sobre la cual se desarrolla el trabajo, de acuerdo a los requerimientos de la actividad

```
npm warn tsd@ENGINE required: { node: '>=12.0.0' },
npm warn tsd@ENGINE current: { node: 'v20.19.0', npm: '10.8.2' }
npm warn tsd@ENGINE }

up to date, audited 1202 packages in 5s
279 packages are looking for funding
  run 'npm fund' for details
found 0 vulnerabilities

Join the Ionic Community! @ ❤️
Connect with millions of developers on the Ionic Forum and get access to live events, news updates, and more.
  Create free Ionic account? No

Your Ionic app is ready! Follow these next steps:
- Go to your new project: cd .\sweet-dreams-app-menu
- Run ionic serve within the app directory to see your app in the browser
- Run ionic capacitor add to add a native iOS or Android project using Capacitor
- Generate your app icon and splash screens using cordova-res --skip-config --copy
- Explore the Ionic docs for components, tutorials, and more: https://ion.link/docs
- Building an enterprise app? Ionic has Enterprise Support and Features: https://ion.link/enterprise-edition
PS C:\>
```

Figura 3. Creación del proyecto sweet-dreams-app
Fuente: Elaboración Propia (2026)

3.2 Estructura inicial de carpetas generada

Accedemos al proyecto desde nuestro editor de código para comprender la estructura generada, esto es fundamental para la entender la función de cada carpeta dentro de nuestra aplicación para la correcta adaptación de acuerdo a lo que se quiere. La plantilla creo tres carpetas (tab1, tab2, tab3), que representan cada una de las pestañas del menú, las cuales serán renombradas conceptualmente y modificadas en su contenido

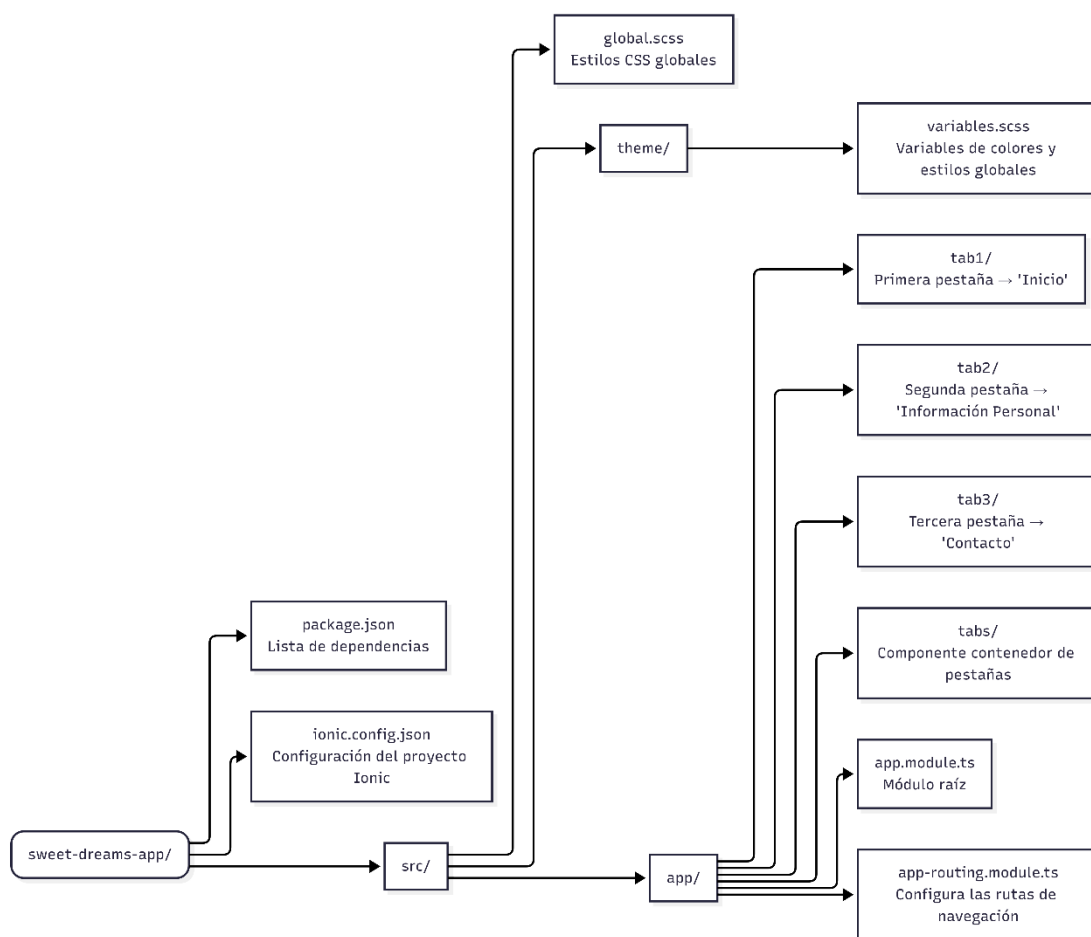


Figura 4. Estructura inicial del proyecto sweet-dreams-app

Fuente: Elaboración Propia (2026)

4. Configuración de los Menús

Como primer paso se renombra la carpeta **tabs** del proyecto como **pages** con el fin de reflejar el propósito claro de la carpeta al alojar las diferentes vistas de la aplicación. Dentro de esta reestructuración realizamos la modificación de la pantalla de Home (inicio): **src/app/pages/home/home.page.html**. Este archivo cumplirá las funciones de pagina principal, donde se incluye un encabezado con un botón para el despliegue de un menú lateral (side bar y el contenido principal los productos.

El elemento `<ion-menu-button color="primary">`, ubicado dentro de `<ion-buttons slot="start">`, es el responsable de abrir el menú lateral. Del lado derecho del encabezado tenemos un botón de carrito (`<ion-icon name="bag-handle-outline">`) y un indicador visual (`<div class="dot">`). El cuerpo de la pagina se encuentra dentro de `<ion-content>` donde se encuentra la sección de bienvenida (hero-section) y un listado de productos generado dinámicamente mediante el componente `<app-product-card>` para cada elemento del flujo `products$ | async`.

```
<!-- PAGINA DE INICIO(HOME) -->

<!-- Header con diseño minimalista y logo dulce -->
<ion-header class="ion-no-border">
  <ion-toolbar>
    <!-- Botón para abrir el menú lateral (colocado a la izquierda) -->
    <ion-buttons slot="start">
      <ion-menu-button color="primary"></ion-menu-button>
    </ion-buttons>
    <!-- Título / Logo de la marca -->
    <ion-title>
      <div class="brand-logo">
        <span class="light">SWEET</span><span class="bold">DREAMS</span>
      </div>
    </ion-title>
    <!-- Botón de carrito a la derecha -->
    <ion-buttons slot="end">
      <ion-button class="cart-btn" routerLink="/cart">
        <ion-icon name="bag-handle-outline"></ion-icon>
        <div class="dot"></div>
      <!-- Punto notificador (simula productos en carrito) -->
    </ion-button>
    </ion-buttons>
  </ion-toolbar>
</ion-header>
```

```

<!-- Contenido principal (Cuerpo de la pagina) con scroll habilitado, [fullscreen]="true" para
que ocupe toda la pantalla -->
<ion-content [fullscreen]="true">
  <!-- Sección de Hero / Bienvenida (Imagen de Fondo con texto)-->
  <div class="hero-section">
    <div class="overlay"></div>
    <div class="hero-content">
      <span class="subtitle">HECHO CON AMOR 🍪</span>
      <h1>Postres <span>Artesanales</span></h1>
      <p>Endulzamos tus momentos más especiales con recetas únicas.</p>
    </div>
  </div>

  <!-- Contenedor principal del listado de productos -->
  <div class="main-container">
    <div class="section-header">
      <h2>Nuestras <span>Delicias</span></h2>
      <ion-button fill="clear" color="primary">Ver todo</ion-button>
    </div>
    <!-- Grid de productos: usa un componente hijo 'app-product-card' -->
    <div class="products-grid">
      <app-product-card
        *ngFor="let product of products$ | async"
        [product]="product"
        (addToCart)="addToCart($event)"
      >
      </app-product-card>
    </div>
  </div>
</ion-content>

```

Figura 5. Código home.page.html
Fuente: Elaboración Propia (2026)

4.1 Reconfiguración completa de rutas

La estructura inicial de la aplicación manejaba las rutas desde el archivo **app.routes.ts** el cual apuntaba a una carga diferida de las rutas de pestañas desde **src/app/tabs/tabs.routes.ts**. Este enfoque asume una estructura organizada en torno al componente **tabs** como contenedor principal de la navegación. Sin embargo, para efectos de este proyecto se toma la decisión de organizar todo en torno a un esquema de rutas propio y explicito, centralizado exclusivamente en el archivo **app-routing.module.ts**. Este archivo es el único responsable de definir todas las rutas; en función de los requerimientos de la actividad. Por lo que el sistema de navegación quedo establecido de la siguiente manera:

Tabla 2. Rutas alojadas en app-routing.module.ts

Ruta	Descripción
home	Página de inicio
personal-info	Página de información personal
contact	Página de contacto
product-detail/:id	Vista de detalle de producto con parámetro dinámico
cart	Página del carrito de compras
" (vacío)	Redirección automática a home

Fuente: Elaboración propia (2026)

Cada una de estas rutas utiliza carga diferida (**lazy loading**) mediante **loadChildren**, importando el módulo correspondiente desde la carpeta **pages**. Lo que proporciona una navegación mas limpia y escalable enfocado a las necesidades específicas de nuestra aplicación. Es importante destacar que el archivo **app.route.ts** queda sin efecto, ya que toda la lógica de enrutamiento se gestiona desde **app-routing.module.ts**.

```
import { NgModule } from '@angular/core';
import { PreloadAllModules, RouterModule, Routes } from '@angular/router';

// Desde aca se definen todas las páginas o vistas que tiene la app
const routes: Routes = [
  {
    path: 'home', // Ruta de Inicio
    loadChildren: () => import('./pages/home/home.module').then( m => m.HomePageModule)
  },
  {
    path: 'personal-info', // Ruta para la informacion personal
    loadChildren: () => import('./pages/personal-info/personal-info.module').then( m =>
m.PersonalInfoPageModule)
  },
];
```

Figura 6. Fragmento Código app.routing.module.ts

Fuente: Elaboración Propia (2026)

5. DESARROLLO DEL CONTENIDO DE CADA PAGINA

5.1. PAGINA DE INICIO (home.page.html)

La estructura y los elementos visuales de la página de inicio fueron analizados en detalle en el apartado 5., donde se describió el renombramiento de la carpeta tabs a pages, la función del botón de menú lateral (`<ion-menu-button>`), el botón de carrito con su indicador visual (`<div class="dot">`), la sección de bienvenida (**hero-section**) y el listado dinámico de productos mediante el componente `<app-product-card>` iterado sobre el flujo `products$ | async`. Por lo tanto, no se repetirá aquí dicha descripción.

5.2. home.page.ts

Este archivo TypeScript actúa como controlador de la página. Para efecto de nuestra pantalla de inicio la lógica requerida es mínima, por lo que el código casi no sufrió cambios de acuerdo a su estructura original. El decorador `@component:` es la etiqueta que le dice a Angular que es un componente donde se indica dentro de el:

- **Selector:** El nombre que tendrá en HTML para llamar a este componente (**app-home**).
- **templateUrl:** la ruta del archivo HTML que le da forma a la página (**home.page.html**).
- **styleUrls:** la ruta del archivo de estilos CSS.

La clase **HomePage** es donde va la lógica. Implementa la interfaz **OnInit**, que obliga a tener un método **ngOnInit** (lo que se ejecuta al cargar la página). Se declara una variable llamada **products\$** (el símbolo \$ indica que es un observable, una especie de "stream" de datos que puede cambiar con el tiempo). Esta variable contendrá la lista de productos.

En el constructor (el método que se ejecuta al crear el componente) se inyectan dos servicios:

- **ProductService**: se encarga de traer los productos.
- **CartService**: se encarga de gestionar el carrito.

En el método `ngOnInit` se le asigna a `products$` el resultado de llamar a `this.productService.getProducts()`, es decir, se carga la lista de productos desde el servicio.

```
// LOGICA DE LA PAGINA HOME
import { Component, OnInit } from '@angular/core';
import { ProductService } from '../../core/services/product.service';
import { CartService } from '../../core/services/cart.service';
import { Product } from '../../core/models/product.model';
import { Observable } from 'rxjs';

// @Component le indica a Angular que la siguiente clase es un componente.
// Define el selector (nombre con el que se usará en HTML), la plantilla (archivo HTML) y los
estilos (archivo SCSS).
@Component({
  selector: 'app-home',
  templateUrl: './home.page.html',
  styleUrls: ['./home.page.scss'],
  standalone: false
})
export class HomePage implements OnInit {
  // Variable para guardar la lista de productos que vienen del servicio
  products$: Observable<Product[]>;
  // Inyectamos los servicios en el constructor.
  constructor(
    private productService: ProductService,
    private cartService: CartService
  ) {}

  ngOnInit() {
    // ngOnInit: se ejecuta cuando la página se inicializa
    this.products$ = this.productService.getProducts();
  }

  // Función que se ejecuta cuando el usuario hace clic en "Agregar al carrito"
  // desde el componente hijo ProductCardComponent.
  addToCart(product: Product) {
    this.cartService.addToCart(product);
    console.log('Producto agregado al carrito:', product.name);
  }
}
```

Figura 7. Código home.page.ts
Fuente: Elaboración Propia (2026)

5.3. home.module.ts

Es la caja contenedora donde se agrupa todo lo que necesita la pagina de inicio.

- **imports:** trae las piezas necesarias como los componentes ionic (ion-header, ion-content), las directivas de Angular (*ngFor) y la configuración de la ruta.
- **declarations:** registra la página **HomePage** y el componente **ProductCardComponet** (que muestra cada producto).
- **providers:** no se usa porque los servicios ya están disponibles de forma global.

```
// MÓDULO QUE ORGANIZA TODO LO NECESARIO PARA EL HOME (PÁGINA DE INICIO).
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { IonicModule } from '@ionic/angular';
import { FormsModule } from '@angular/forms';
import { HomePage } from '../home.page';
import { RouterModule } from '@angular/router';
import { ProductCardComponent } from '../../shared/components/product-card.component';

@NgModule({
  imports: [
    CommonModule, // Necesario para directivas básicas
    FormsModule, // Para trabajar con formularios
    IonicModule, // Componentes de Ionic (ion-header, ion-content, etc.)
    RouterModule.forChild([ // Rutas específicas de este módulo (carga diferida)
      {
        path: '', // Ruta base del home (ej: /home)
        component: HomePage // HomePage
      }
    ])
  ],
  declarations: [HomePage, ProductCardComponent] // Los componentes de este módulo
})
export class HomePageModule {}
```

Figura 8. Código home.module.ts
Fuente: Elaboración Propia (2026)

5.4. home.page.scss

El archivo **home.page.scss** emplea las siguientes clases para estructurar la página: **.brand-logo** (con sus subelementos **.light** y **.bold**), **.cart-btn** y su notificador **.dot**, la sección **.hero-section** con **.overlay** y **.hero-content** (dentro: **.subtitle**, **h1**, **p**), el contenedor **.main-container** con el encabezado **.section-header** y **h2**, y finalmente el grid **.products-grid**. Desde acá se maneja la apariencia visual del home y se definen los estilos del logo, el botón del carrito y la imagen de fondo.

5.5. PAGINA DE INFORMACION PERSONAL (personal-info.html)

Desde esta página abordamos el requerimiento de nuestra actividad en relación a la página de información personal. Es aquí donde se muestra la información personal del chef pastelero (Perfil). Lo estructuramos de la siguiente manera:

- **<ion-header>**: Encabezado de la página
- **<ion-toolbar color="primary">**: Barra superior con color primario
- **<ion-menu-button>**: Botón para abrir el menú lateral
- **<ion-title>**: Título de la página ("Sobre la Chef")
- **<ion-content [fullscreen]="true">**: Cuerpo principal con scroll a pantalla completa
- **<ion-avatar>**: Contenedor de imagen circular (avatar de la chef)
- ****: Foto de perfil
- **<h2>**: Nombre de la chef (con interpolación `{{ nombre }}`)
- **<p> (tagline)**: Carrera o subtítulo (`{{ carrera }}`)
- **<div class="bio-container">**: Biografía (`{{ descripcion }}`)
- **<div class="skills-section">**: Conocimientos y especialidades
- ***ngFor="let habilidad of habilidades"**: Bucle para mostrar cada habilidad
- **<div class="skill-pill">**: Etiqueta visual de cada especialidad
- **<ion-icon name="quote">**: Icono de comillas decorativas


```

<!-- INFORMACION PERSONAL: PERFIL DEL CHEF PASTELERO -->

<!-- Encabezado de la pagina sin borde, color primario, con botón de
menú -->
<ion-header class="ion-no-border">
  <ion-toolbar color="primary">
    <ion-buttons slot="start">
      <ion-menu-button></ion-menu-button>
      <!-- Abre el menu lateral -->
    </ion-buttons>
    <!-- Título de la pagina -->
    <ion-title>Sobre la Chef</ion-title>
  </ion-toolbar>
</ion-header>
<!-- Contenido principal con scroll completo -->
<ion-content [fullscreen]="true">
  <div class="chef-profile">
    <!-- Bloque superior con fondo degradado y el avatar -->
    <div class="profile-header">
      <div class="header-overlay"></div>
      <!-- Avatar circular con foto de la chef (la misma del menú) -->
      <ion-avatar class="chef-avatar">
        <!-- Foto de Perfil -->
        
      </ion-avatar>
    </div>
  </div>
</ion-content>

```

Figura 9. Fragmento Código personal-info.html
Fuente: Elaboración Propia (2026)

5.6. personal-info.page.ts

Desde acá se define la lógica de la página de información personal. Con cuatro propiedades públicas: **nombre**, **carrera**, **descripción** (manejando datos estáticos de la Chef) y un **array** con cuatro especialidades que definen las habilidades del chef. El método **ngOnInit** solo registra un mensaje en consola. No se inyectan servicios ni hay lógica adicional, ya que la página es específicamente informativa.

```

// Lógica de la página Sobre la Chef.
import { Component, OnInit } from '@angular/core';
// @Component le indica a Angular que la siguiente clase es un
componente.
// Define el selector (nombre con el que se usará en HTML), la
plantilla y estilo
@Component({
  selector: 'app-personal-info',
  templateUrl: './personal-info.page.html',
  styleUrls: ['./personal-info.page.scss'],
  standalone: false
})
export class PersonalInfoPage implements OnInit {
  // Datos personales que se muestran en la plantilla
  nombre: string = 'Iliana Coronado';
  carrera: string = 'Chef Pastelera & Repostera';
  descripcion: string = 'Especialista en repostería creativa y diseño
de postres de autor. Con más de 5 años endulzando los momentos más
especiales de mis clientes.';

  // Conocimientos específicos para la temática
  habilidades: string[] = [
    'Decoración con Fondant',
    'Técnicas de Repostería Francesa',
    'Diseño de Pasteles de Boda',
    'Manejo de Chocolatería Fina'
  ];

  constructor() { }
  // ngOnInit: ciclo de vida que se ejecuta al iniciar la página

  ngOnInit() {
    console.log('Perfil de la Chef Iliana cargado');
  }
}

```

Figura 10. Código personal-info.ts
Fuente: Elaboración Propia (2026)

5.7. personal-info.module.ts

El archivo **personal-info.module.ts** es un módulo de carga diferida (se importa mediante **loadChildren** desde **app-routing.module.ts**). No tiene lógica adicional, ni servicios propios tampoco componentes declarados aparte.

```
// MODULO QUE ORGANIZA LA INFORMACION REFERENTE AL CHEF.
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { IonicModule } from '@ionic/angular';
import { PersonalInfoPage } from './personal-info.page';
import { RouterModule } from '@angular/router';

@NgModule({
  imports: [
    CommonModule,
    FormsModule,
    IonicModule,
    RouterModule.forChild([
      {
        path: '', // Ruta base: /personal-info
        component: PersonalInfoPage
      }
    ])
  ],
  declarations: [PersonalInfoPage] // Solo este componente
})
export class PersonalInfoPageModule {}
```

Figura 11. Código personal-info.module.ts
Fuente: Elaboración Propia (2026)

5.8. personal-info.page.scss

Desde acá se estiliza el perfil de la página de información personal:

.profile-header con fondo degradado y avatar circular **.chef-avatar** que sobresale; **.profile-body** contiene **.chef-name**, **.chef-tagline**, **.bio-container** (tarjeta blanca), **.skills-section** con **.skills-grid** y etiquetas **.skill-pill**, y **.quotes-container** con icono y cita. Mediante estos estilos se consigue un perfil visualmente atractivo, destacando el avatar destacado, biografía en tarjeta, especialidades etiquetadas y una frase inspiradora, con una paleta basada en colores primarios y secundarios del tema.

5.9. PAGINA DE CONTACTO: contac.page.html

Coordenadas de contacto y formulario para pedidos especiales su estructura incluye: un encabezado (<ion-header>) con menú lateral y título "Pedidos Especiales"; un cuerpo (<ion-content>) con una cuadrícula responsiva (<ion-grid>, <ion-row>, <ion-col>). La columna izquierda (size-md="5") muestra redes sociales (Instagram, WhatsApp, correo electronico) usando <ion-item> con íconos y etiquetas. La columna derecha (size-md="7") contiene un formulario con campos: nombre (<ion-input>), tipo de evento (<ion-select> con opciones), detalles (<ion-textarea>) y un botón de envío (<ion-button>).

```
<!-- PAGINA DE CONTACTOS Y PEDIDOS ESPECIALES -->
<!-- header sin borde, con toolbar color primario -->
<ion-header class="ion-no-border">
  <ion-toolbar color="primary">
    <!-- Botón para abrir el menú lateral -->
    <ion-buttons slot="start">
      <ion-menu-button></ion-menu-button>
    </ion-buttons>
    <ion-title>Pedidos Especiales</ion-title>
  </ion-toolbar>
</ion-header>
<!-- Contenido principal con padding general -->
<ion-content class="ion-padding">
  <!-- Bloque de bienvenida / hero -->
  <div class="contact-hero">
    <h1>Haz tu evento inolvidable 🍰</h1>
    <p>Diseñamos postres personalizados para tus fechas más importantes.</p>
  </div>
  <!-- Grid responsivo: en móvil 1 columna, en tablet/desktop 2 columnas -->
  <ion-grid>
    <ion-row>
      <!-- Redes Sociales (Izquierda) -->
      <ion-col size="12" size-md="5" class="social-column">
        <div class="info-card">
          <h3>¡Contactanos!</h3>
          <p>Síguenos en nuestras redes o escríbenos directamente.</p>
          <!-- Instagram -->
          <ion-item lines="none" class="social-item">
            <ion-icon
```

Figura 12. Fragmento Código contac.page.html

Fuente: Elaboración Propia (2026)

5.10 contact.page.ts

Contiene la lógica de la página de contacto. Se declara un objeto contacto con propiedades **nombre**, **email** y **mensaje**. En **ngOnInit** solo registra un mensaje en consola. Define el método **enviarFormulario()** que valida si los tres campos están llenos; si es así, muestra una alerta y limpia el objeto; de lo contrario, alerta pidiendo completar los campos.

```
// LOGICA DE LA PAGINA DE CONTACTO.
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-contact',
  templateUrl: './contact.page.html',
  styleUrls: ['./contact.page.scss'],
  standalone: false
})
export class ContactPage implements OnInit {
  // Objeto que guarda los datos del formulario
  contacto = {
    nombre: '',
    email: '',
    mensaje: ''
  };

  constructor() { }
  // ngOnInit: se ejecuta al iniciar la página
  ngOnInit() {
    console.log('Contacto iniciado');
  }
  // Función que se llamaría al enviar el formulario (por ahora no está
  // enlazada al HTML)
  enviarFormulario() {
    // Validación básica: si todos los campos tienen contenido
    if (this.contacto.nombre && this.contacto.email &&
    this.contacto.mensaje) {
      alert('¡Mensaje enviado!');
      // Limpiar formulario después de enviar
      this.contacto = { nombre: '', email: '', mensaje: '' };
    } else {
      alert('Por Favor Complete los campos,');
    }
  }
}
```

Figura 13. Fragmento Código contact.page.ts

Fuente: Elaboración Propia (2026)

5.11. contact.module.ts

Este archivo al igual que los archivos anteriores de su tipo es un modulo de carga diferida se importa mediante **loadChildren** desde **app-routing.module.ts**. En imports incluye **RouterModule.forChild** para definir la ruta base asociada a **/contact** junto con **CommonModule**, **FormsModule** (necesario para el enlace de datos del formulario) e **IonicModule**. Declara únicamente **ContactPage** y no incluye **providers** porque los servicios se inyectan a nivel raíz.

```
// MODULO PARA LA CONFIGURACION DE LA PAGINA DE CONTACTO Y PEDIDOS ESPECIALES.
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { IonicModule } from '@ionic/angular';
import { ContactPage } from './contact.page';
import { RouterModule } from '@angular/router';

@NgModule({
  imports: [
    CommonModule,
    FormsModule, // Necesario para enlazar datos del formulario
    IonicModule,
    RouterModule.forChild([{ path: '', component: ContactPage }]) //
    Ruta base: /contact
  ],
  declarations: [ContactPage] // Solo este componente en el módulo
})
export class ContactPageModule {}
```

Figura 14. Código contact.module.ts
Fuente: Elaboración Propia (2026)

5.12. contact.page.scss

El archivo **contact.page.scss** define los estilos de la página de contacto. Principalmente: **.contact-hero** (encabezado con degradado y texto centrado); **.social-column** con **.info-card** (tarjeta blanca para redes sociales, con **.social-item** para cada contacto y **.emoji-**

deco decorativo; **.form-container** (tarjeta blanca del formulario, con **.sweet-input** para personalizar campos, y **.send-btn** para el botón de envío). Incluye además un **@media** para ajustar márgenes en móviles.

5.13. Product-detail

La página de detalle de producto (**product-detail**) constituye un valor agregado fundamental, ya que permite al usuario visualizar información completa de un artículo antes de agregarlo al carrito, mejorando la toma de decisión. La vista (**product-detail.page.html**) presenta un encabezado con botón de retroceso y, mediante ***ngIf**, muestra la imagen destacada y una hoja inferior (**info-sheet**) con categoría, nombre, calificación, descripción, tiempo de elaboración, precio y un botón “Agregar al Carrito” que ejecuta **addToCart()**.

5.14. cart

El módulo **cart** gestiona el carrito de compras: la vista lista productos con imagen, cantidad y precio, permite eliminar deslizando y muestra el total con opción de finalizar compra (simulada). La lógica usa observables del **CartService** para actualizar automáticamente los ítems y el total, con estilos que incluyen mensaje de carrito vacío y tarjetas redondeadas.

6. PALETA Y ESTILOS GLOBALES

6.1. Modificación del archivo **variables.scss**

Desde este archivo **ionic** define sus colores mediante variables CSS (**--ion-color***). Su modificación afecta automáticamente a todos los componentes que las utilicen. Para este proyecto buscamos una identidad, mediante la personalización de los siguientes colores:

Tabla 3. Paleta de colores

Rol CSS	Valor	Uso principal
--ion-color-primary	#FF85A1 (rosa pastel)	Botones, iconos, acentos, texto de títulos
--ion-color-secondary	#FCE4EC (crema/vainilla)	Fondos degradados, detalles decorativos
--ion-background-color	#FFF9FB (crema)	Fondo general de la app
--ion-text-color	#5D4037 (marrón café)	Texto principal
--soft-shadow	0 10px 30px rgba(255,133,161,0.1)	Sombras suaves en tarjetas
--accent-gold	#FFB300	Acentos (ej. estrellas de calificación)
--text-light	#8D6E63	Texto secundario o de menor énfasis

Fuente: Elaboración Propia (2026)

Es importante acotar que se desactivo el modo oscuro y se aplicaron estilos globales a `ion-toolbar`, `ion-title` e `ion-card` con el fin de mantener coherencia.

```

/* Configuración de colores Armoniosos "Sweet Dreams" */
:root {
  /** primario: Rosa Pastel Elegante **/
  --ion-color-primary: #FF85A1;
  --ion-color-primary-rgb: 255, 133, 161;
  --ion-color-primary-contrast: #ffffff;
  --ion-color-primary-contrast-rgb: 255, 255, 255;
  --ion-color-primary-shade: #e0758e;
  --ion-color-primary-tint: #ff91aa;

  /** secundario: Crema / Vainilla **/
  --ion-color-secondary: #FCE4EC;
  --ion-color-secondary-rgb: 252, 228, 236;
  --ion-color-secondary-contrast: #3E2723;
  --ion-color-secondary-contrast-rgb: 62, 39, 35;

  /** Fondos y Textos **/
  --ion-background-color: #FFF9FB;
  --ion-background-color-rgb: 255, 249, 251;

  /*
    DISEÑO ATÓMICO: Paleta boutique Sweet Dreams.
    Marrón Café para calidez y Rosa Pastel para dulzura.
  */
  --ion-text-color: #5D4037;
  --ion-text-color-rgb: 93, 64, 55;

```

Figura 15. Fragmento Código Variables.scss

Fuente: Elaboración Propia (2026)

7. EJECUCION Y PRUEBAS DE LA APLICACIÓN

7.1. Inicio desde el servidor de desarrollo

Una vez que finaliza la implementación del código, se procede a correr la aplicación desde el servidor de desarrollo de ionic con el fin de verificar el correcto funcionamiento de la aplicación en el navegador. Esto se lleva a cabo mediante el comando **ionic serve**. Este comando compila el código TypeScript a JavaScript, levanta un servidor local y abre el navegador en la dirección indicada <http://localhost:8100/home>

```
PS C:\proyectosionic> cd sweet-dreams-app
PS C:\proyectosionic\sweet-dreams-app> ionic serve
```

Figura 16. Comando ionic serve para correr el servidor de desarrollo
Fuente: Elaboración Propia (2026)

Una funcionalidad importante de destacar es la recarga en caliente (**live reload**), la cual permite que tras la modificación y guardado de un archivo de la aplicación, el navegador actualice la vista de manera automática, sin necesidad de reiniciar el servidor. Esto optimiza de manera significativa el flujo de desarrollo y pruebas al reducir los tiempos de espera y facilita la iteración continua.

```
Use Ctrl+C to quit this process

[INFO] Browser window opened to http://localhost:8100!

[ng] ✓ Browser application bundle generation complete.
[ng] Initial chunk files | Names | Raw size
[ng] runtime.js | runtime | 13.12 kB |
[ng] 69 unchanged chunks
[ng] Build at: 2026-05-07T11:02:37.670Z - Hash: b22499d9ba5640bc - Time: 18576ms
[ng] ✓ Compiled successfully.
[ng] ✓ Browser application bundle generation complete.
[ng] 70 unchanged chunks
[ng] Build at: 2026-05-07T11:02:39.271Z - Hash: b22499d9ba5640bc - Time: 1316ms
[ng] ✓ Compiled successfully.
```

Figura 17. Compilación exitosa servidor corriendo a través del puerto 8100
Fuente: Elaboración Propia (2026)

7.2. Prueba de adaptabilidad a dispositivos móviles (responsive)

Con el fin de verificar la adaptabilidad de la aplicación en dispositivos móviles y que todos sus elementos se adapten de forma responsive, utilizamos el modo de simulación móvil de Chrome DevTools. De acuerdo a los siguientes pasos:

1. Con la aplicación corriendo en el navegador Chrome en la dirección <http://localhost:8100/home>
2. Presionar la tecla **F12**, para abrir las herramientas de desarrollo (**Chrome DevTools**)
3. Hacer clic en el ícono **Toggle device toolbar**, que representa un teléfono móvil y una tableta. También se puede usar el atajo de teclado Ctrl+Shift+M (en Windows/Linux) o Cmd+Shift+M (en Mac). Este ícono se encuentra en la esquina superior izquierda de la ventana de DevTools, junto a la pestaña "Elements".
4. Activada la vista de dispositivo, seleccionar un modelo concreto de la lista desplegable. Esto ajusta la resolución y las proporciones de la pantalla para simular ese modelo de celular.
5. Probar la navegación completa de la aplicación en este modo, verificando el comportamiento de las páginas:
 - a. **Inicio (home)**: visualización correcta del logotipo, botones, sección de bienvenida y cuadrícula de productos.
 - b. **Sobre la Chef (personal-info)**: adaptación del avatar, biografía y etiquetas de especialidades.
 - c. **Pedidos Especiales (contact)**: disposición de los campos del formulario y los datos de contacto en una sola columna (apilados).
6. Se comprueba la correcta redimensión y organización de todos los elementos, sin desbordamientos ni solapamientos, demostrando que la aplicación es plenamente responsiva y apta para uso en dispositivos móviles.

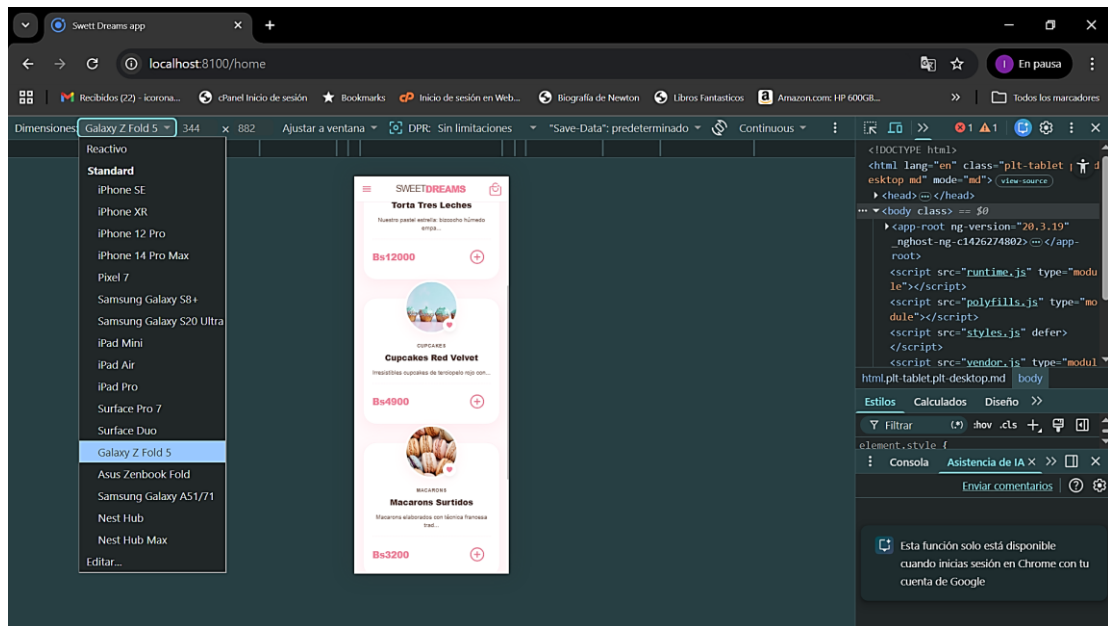


Figura 18. Paso a paso para simular móviles en DevTools
Fuente: Elaboración Propia (2026)

7.3. Verificación de Funcionamiento

Para confirmar que la aplicación cumple con los requerimientos de la actividad se llevaron a cabo las siguientes pruebas:

Tabla 4. Verificación de Funcionamiento

Prueba realizada	Resultado esperado	Estado
La app abre y carga la página de inicio (Home)	Se muestra el logotipo "SWEET DREAMS", botón de menú lateral (☰), botón de carrito (🛒), sección de bienvenida con imagen de fondo y el listado de productos disponibles.	 Correcto
Accede a la sección "Sobre la Chef" desde el menú lateral	Se visualiza el avatar de la chef, su nombre (Iliana Coronado), carrera (Chef Pastelera & Repostera), biografía detallada y la lista de especialidades (Decoración con Fondant, Técnicas Francesas, etc.).	 Correcto
Accede a la sección de "Pedidos Especiales" desde el menú lateral	Se muestra el formulario de cotización con campos: Nombre, Tipo de evento, Detalles del pedido, además de los datos de contacto (Instagram, WhatsApp, correo electrónico).	 Correcto
Seleccionar un producto desde la pagina de Inicio	Se redirige correctamente a la vista de detalle (/product-detail/:id), mostrando imagen ampliada, nombre, categoría, calificación, descripción, tiempo de elaboración, precio y botón "Agregar al Carrito".	 Correcto
Agregar un producto al carrito desde la vista detalle	El producto se añade al carrito (se verifica en la página /cart).	 Correcto
Ingresar a la pagina del carrito	Se listan los productos añadidos con imagen, nombre, cantidad, precio unitario y subtotal. Se puede eliminar un producto deslizando hacia la izquierda (opción con icono de papelera).	 Correcto
Finalizar la compra desde el carrito	Al pulsar el botón "Finalizar Compra", se muestra una alerta simulada "¡Gracias por tu compra! Tu pedido está en camino."	 Correcto

Fuente: Elaboración Propia (2026)

7.4. Visualización de las vistas de la aplicación

Se accede a la aplicación desde la dirección <http://localhost:8100>, la carga inicial va hacia la pagina **home**, dónde se muestra un encabezado con el logotipo de la marca SWEET DREAMS”, se muestra el botón para desplegar el menú lateral del lado izquierdo y un botón de acceso rápido al carrito de compra del lado derecho. Una sección de bienvenida con imagen de fondo y debajo se listan los productos disponibles.

Desde el menú latera se accede a las paginas "Sobre la Chef" (**personal-info**), que presenta la biografía, especialidades y una fotografía de la pastelera; y hacia la página "Pedidos Especiales" (**contact**), que incluye un formulario de cotización y los datos de contacto (Instagram, WhatsApp, correo).

Al seleccionar un producto desde el listado de inicio, se accede a la vista de detalle (**product-detail**) con la imagen ampliada, descripción, precio y un botón para agregar al carrito. Finalmente, la página del carrito (**cart**) resume los productos añadidos, permite eliminar artículos deslizando y muestra el total a pagar, con un botón simulado de finalización de compra. Todas las vistas mantienen una paleta cromática consistente (tonos pastel y marrón café) y ofrecen una experiencia de navegación fluida gracias a la recarga en caliente del servidor de desarrollo.

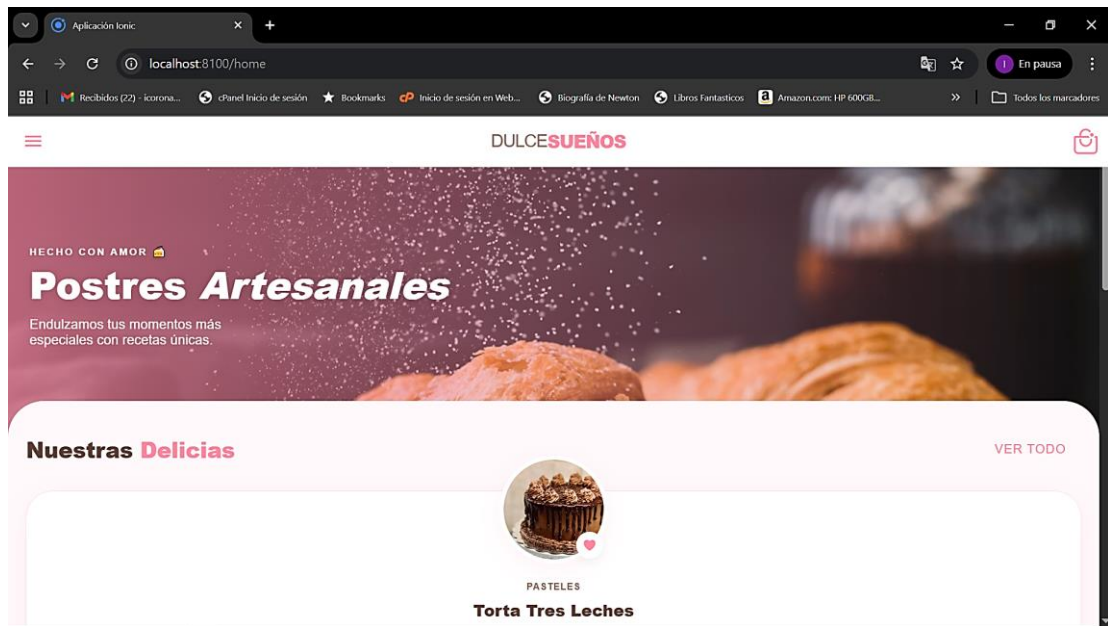


Figura 19. Página Home (Inicio)
Fuente: Elaboración Propia (2026)

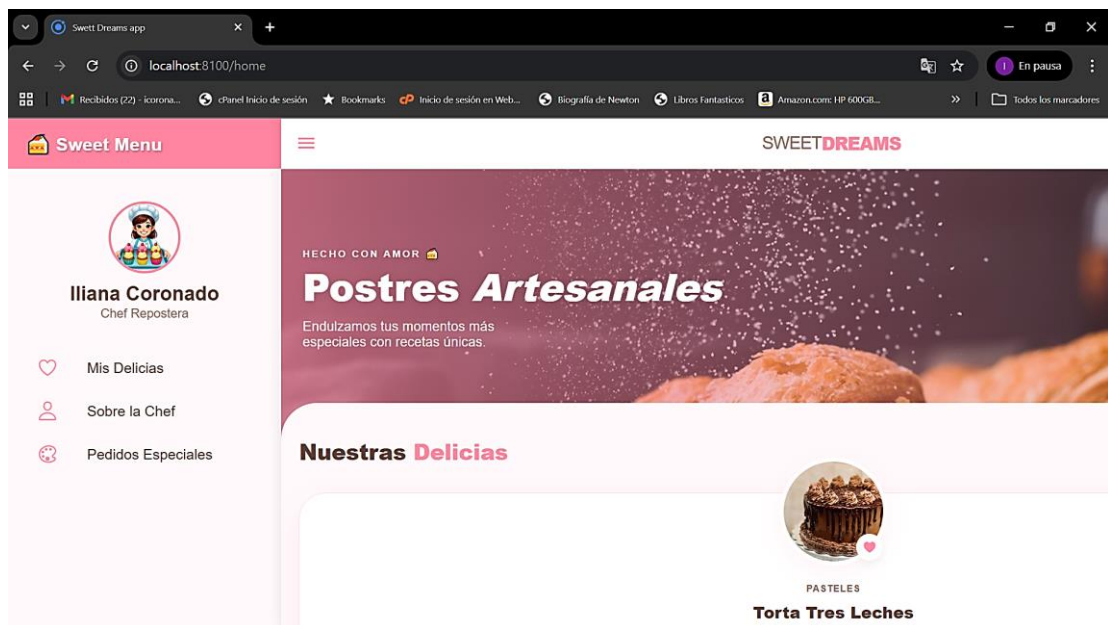


Figura 20. Página Home muestra menú lateral
Fuente: Elaboración Propia (2026)

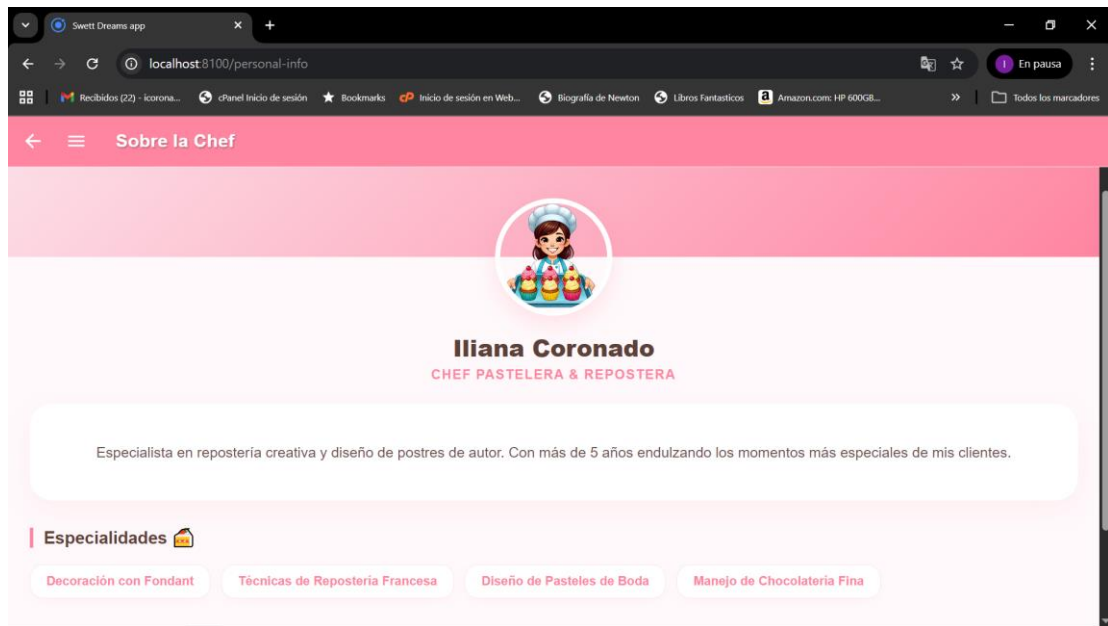


Figura 21 Página Sobre la Chef (personal-info.html)
Fuente: Elaboración Propia (2026)

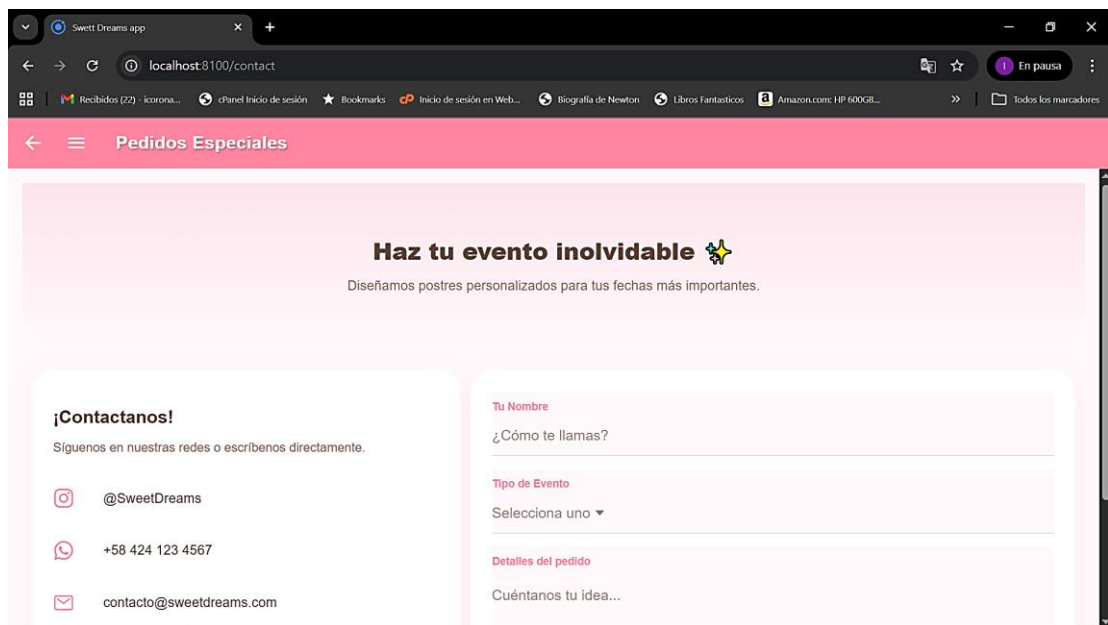


Figura 22. Página Pedidos Especiales (contact.html)
Fuente: Elaboración Propia (2026)

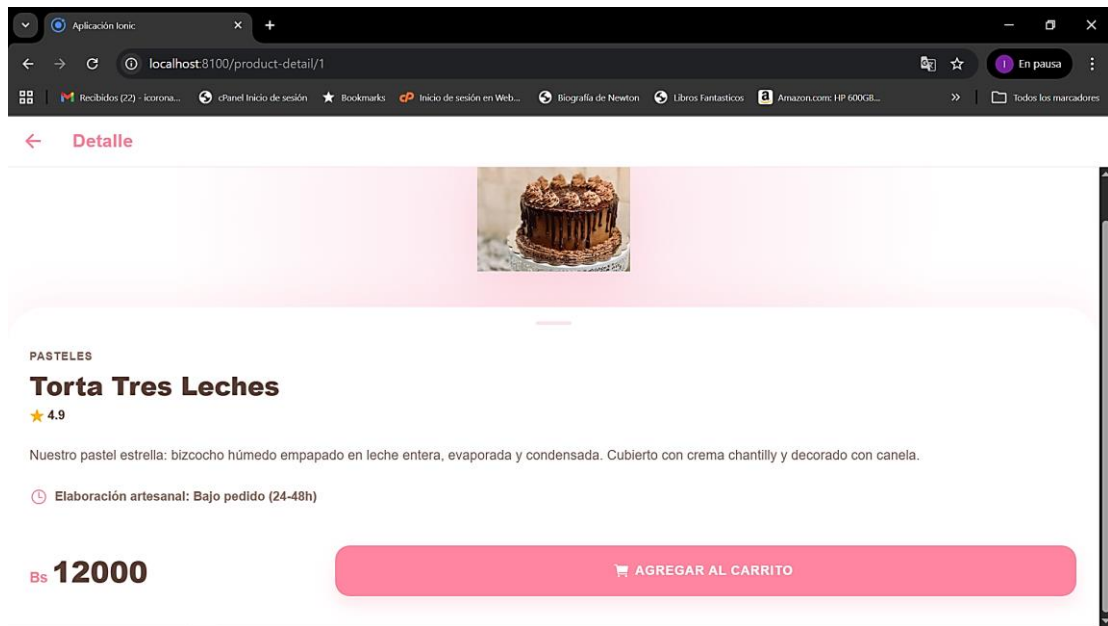


Figura 23. Página Detalle (product-detail.html)
Fuente: Elaboración Propia (2026)

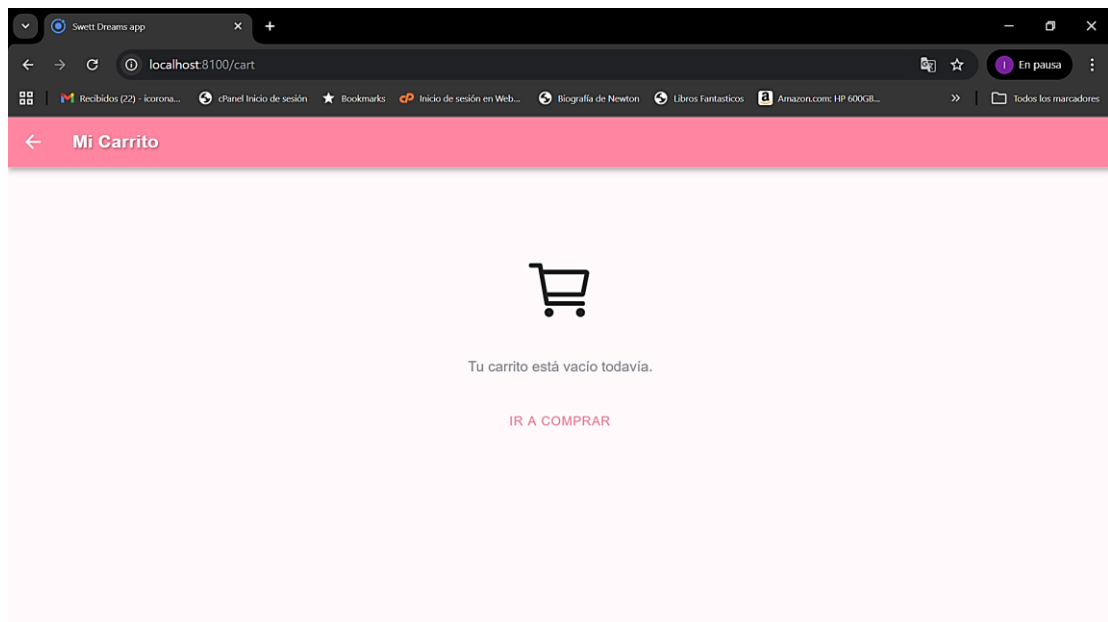


Figura 24. Página Mi Carrito (cart.html)
Fuente: Elaboración Propia (2026)

8. Disponibilidad del Proyecto en GitHub

El proyecto completo está disponible en un repositorio público de GitHub, lo que facilita su acceso y revisión. El desarrollo del código fuente contó con el apoyo de herramientas de inteligencia artificial generativas, específicamente Claude AI y Deepseek. El código resultante fue revisado, validado e integrado manualmente por la desarrolladora del proyecto. Pueden acceder mediante el siguiente enlace: <https://github.com/icoronado48-tech/sweet-dreams-app.git>

CONCLUSIONES

El desarrollo de la aplicación **sweet-dreams-app** ha demostrado la viabilidad de construir una aplicación móvil híbrida funcional y responsiva utilizando Ionic Framework junto con Angular, partiendo de una plantilla base. A lo largo del proceso, se consolidaron conceptos fundamentales como la organización modular, la inyección de dependencias, el enrutamiento con carga diferida y el uso de observables (RxJS), lo que permitió crear una arquitectura escalable y mantenible. La aplicación resultante cumple con todos los requerimientos funcionales de la actividad, ofreciendo una experiencia de navegación fluida y una estética visual coherente con la identidad de la marca.

Entre los logros técnicos más relevantes se destacan: la correcta configuración del entorno de desarrollo (Node.js, Ionic CLI, Angular), la reestructuración completa del sistema de rutas, reemplazando el esquema original basado en tabs por un enfoque centralizado en **app-routing.module.ts** y la implementación de páginas clave (**home**, **personal-info**, **contact**, **producto-detail** y **cart**). Además, se integraron servicios reutilizables (**ProductService**, **CartService**), componentes compartidos (**ProductCardComponent**) y una paleta de colores personalizada desde **variables.scss**, que refuerza la identidad gráfica. Las pruebas de adaptabilidad con **Chrome DevTools** confirmaron el comportamiento responsivo de la interfaz en distintos dispositivos móviles. Finalmente, el proyecto incorpora valor agregado mediante una vista de detalle de producto y un carrito de compras (con lógica simulada), acercando la aplicación a un escenario real de comercio electrónico.

REFERENCIAS

- **Angular Team. (2024).** Angular — Template-driven forms. Recuperado de <https://angular.io/guide/forms>
- **Anthropic (2026).** Claude AI (Versión Sonnet 4) [Modelo de lenguaje grande]. <https://claude.ai>
- **DeepSeek (2026).** DeepSeek (Versión V4) [Asistente de inteligencia artificial]. <https://chat.deepseek.com>
- **Google. (2024).** Chrome DevTools — Simulate mobile devices with device mode. Recuperado de <https://developer.chrome.com/docs/devtools/device-mode>
- **Ionic Framework. (2024).** Ionic Documentation — Building your first app. Recuperado de <https://ionicframework.com/docs/intro/cli>
- **Ionicons. (2024).** Ionicons — Premium designed icons for use in web, iOS, Android, and desktop apps. Recuperado de <https://ionic.io/ionicons>
- **Node.js Foundation. (2024).** Node.js — About. Recuperado de <https://nodejs.org/en/about>